

Guia per al desplegament de KarmaWebApi mitjançant Docker

Contingut

1.	Introducció	3
1.1.	Enfocament adoptat: backend com a servei	3
1.2.	I per què Docker?	3
2.	Paquet “Docker” per a l’alumnat.....	4
2.1.	Estructura.....	4
2.2.	Descripció dels fitxers inclosos	4
2.3.	Funcionament des del punt de vista de l’alumne	5
3.	Descripció tècnica del sistema Docker	5
3.1.	docker-compose.yml	5
	Configuració del servei de base de dades.....	6
	Configuració del servei de l’API.....	6
	Volums i persistència de dades	7
3.2.	Dockerfile	8
	Execució de l’API a partir de binaris (.dll)	8
	Consideracions sobre el procés de build.....	9
3.3.	Gestió de la base de dades	9
	Ús del backup inicial (.bak)	9
	Procés de restauració inicial	9
	Persistència de dades amb volums Docker	10
	Actualització del backup.....	10
4.	Responsabilitats del professorat	10
4.1.	Modificació del backend	10
4.2.	Actualització de la base de dades.....	11
4.3.	Preparació i distribució del paquet.....	11
5.	Ús alternatiu amb Visual Studio.....	12
5.1.	Descripció del projecte amb codi font.....	12

5.2.	Execució en entorn de desenvolupament	13
5.3.	Connexió amb SQL Server.....	13
5.4.	Diferències respecte a l'entorn Docker.....	13
6.	Possibles millores i ampliacions a futur	13
6.1.	Automatització del procés d'inicialització	13
6.2.	Scripts simplificats per a l'alumnat	14
6.3.	Empaquetat en màquina virtual.....	14
7.	Annexos.....	15
7.1.	Problemes habituals i solucions	15
	L'API no s'inicia	15
	Error de connexió amb la base de dades.....	15
	Problemes en la restauració inicial	15
	Problemes de ports (1433)	15
	Estat inconsistent del sistema.....	16
7.2.	Comandes útils de Docker	16
	Arrancada i parada del sistema	16
	Reinici de serveis	16
	Consulta de contenidors.....	16
	Consulta de logs	16
	Execució de comandes dins del contenidor	16
	Neteja de l'entorn.....	17

1. Introducció

En aquesta guia es descriu com s'ha utilitzat Docker en el projecte Karma per a que els alumnes de Desenvolupament Web en Entorn Client puguin utilitzar el backend i la base de dades sense necessitat de realitzar instal·lacions complexes en els seus entorns de treball.

Amb aquest plantejament, es busca reduir les dificultats tècniques inicials associades a la configuració del servidor i centrar l'activitat de l'alumnat en les competències específiques del mòdul.

Per aconseguir-ho, es proporciona un sistema backend totalment preparat per a la seua execució amb una base de dades amb dades inicials, de manera que l'alumnat només hagi de posar-lo en marxa i començar a treballar amb ell.

1.1. Enfocament adoptat: backend com a servei

La solució es basa en proporcionar el backend a l'alumnat ja desplegat i operatiu, de manera similar al que ocorre en entorns professionals. El sistema encapsulat inclou:

- una API funcional
- una base de dades associada

D'aquesta manera, es simplifica notablement el procés de posada en marxa i es garanteix que tots els alumnes treballen sobre una mateixa base.

A nivell didàctic, aquest enfocament permet reproduir una situació real en què el frontend interactua amb un servei extern, afavorint així una millor comprensió de l'arquitectura client-servidor.

1.2. I per què Docker?

L'ús de Docker en aquest projecte respon a la necessitat de crear un entorn de treball estable, reproduïble i independent de la configuració dels equips de l'alumnat.

Docker, permet encapsular tant la base de dades com l'API dins de contenidors que es poden executar de manera uniforme en diferents sistemes operatius (Windows, macOS o Linux), evitant problemes habituals com incompatibilitats, versions incorrectes o configuracions incompletes. El procés d'instal·lació es simplifica molt.

2. Paquet “Docker” per a l’alumnat

2.1. Estructura

El paquet es distribueix com una carpeta disponible a github amb una estructura organitzada, de la següent manera:

- Fitxers de configuració Docker (docker-compose.yml i Dockerfile)
- Carpeta amb el backup de la base de dades
- Carpeta amb el backend compilat
- Documentació per a l’alumnat

Aquesta organització permet identificar fàcilment cada element i entendre el seu paper dins del sistema.

2.2. Descripció dels fitxers inclosos

A continuació es descriu la funció dels principals components del paquet:

◆ **docker-compose.yml**

És el fitxer encarregat de definir i coordinar tots els serveis del sistema.

Permet executar tant la base de dades com l’API amb una única comanda, gestionant també la comunicació entre ambdós.

◆ **Dockerfile**

Defineix el comportament del contenidor de l’API.

En aquest cas, s’utilitza per executar una versió del backend ja compilada, sense necessitat de dur a terme el procés de compilació en el moment de l’execució.

◆ **Carpeta backup**

Conté els recursos relacionats amb la base de dades inicial:

- una còpia de seguretat de la base de dades (fitxer .bak)
- un script que permet restaurar-la per línia de comandos

Aquestes dades s’utilitzen per generar un estat inicial comú per a tot l’alumnat.

◆ **Carpeta api**

Inclou el backend ja compilat en format executable.

Aquesta carpeta conté tots els fitxers necessaris per executar l’API, sense exposar el codi font del projecte.

Per tant, des del punt de vista de l'alumne, aquesta part funciona com una "caixa negra" que proporciona serveis mitjançant una API.

◆ Fitxer README (per a visualitzar a github)

Inclou les instruccions bàsiques perquè l'alumnat pugui posar en marxa el sistema i començar a treballar.

Està dissenyat amb un enfocament pràctic i simplificat.

2.3. Funcionament des del punt de vista de l'alumne

Des del punt de vista de l'alumnat, el sistema funciona de manera molt senzilla.

Una vegada descarregat el paquet i instal·lat Docker, només cal executar unes poques comandes per a posar en marxa el backend.

La primera vegada és necessari restaurar la base de dades a partir del backup proporcionat. A partir d'aquest moment, el sistema manté les dades creades i no cal repetir aquest pas a no ser que es vulgui començar de nou des de 0.

L'alumne pot accedir a l'API mitjançant el navegador i utilitzar-la des del seu propi desenvolupament client.

Aquest funcionament evita que l'alumnat hagi de preocupar-se per tasques d'administració o configuració del servidor.

3. Descripció tècnica del sistema Docker

3.1. docker-compose.yml

El fitxer **docker-compose.yml** és l'element central que permet definir i executar el sistema complet mitjançant Docker.

La seua funció principal és descriure els serveis que formen part de l'aplicació i com aquests interactuen entre si, permetent la seua posada en marxa amb una única ordre.

En aquest projecte, el fitxer defineix dos serveis diferenciats:

- un servei per a la base de dades (SQL Server)
- un servei per a l'API (KarmaWebAPI)

A més, estableix la configuració necessària perquè ambdós puguin comunicar-se i funcionar de manera coordinada.

```

1 services:
2   db:
3     image: mcr.microsoft.com/mssql/server:2022-latest
4     container_name: sqlserver
5     environment:
6       SA_PASSWORD: "Password123!"
7       ACCEPT_EULA: "Y"
8     ports:
9       - "1433:1433"
10    volumes:
11      - ./backup:/var/opt/mssql/backup
12      - sql_data:/var/opt/mssql
13
14    api:
15      build:
16        context: .
17        dockerfile: Dockerfile
18      container_name: karma-api
19      ports:
20        - "5000:5100"
21      depends_on:
22        - db
23
24  volumes:
25    sql_data:
26

```

Figura 1 Contingut de docker-compose.yml

Configuració del servei de base de dades

El servei de base de dades utilitza una imatge oficial de SQL Server distribuïda per Microsoft.

Aquest servei està configurat per:

- executar un servidor de base de dades complet dins d'un contenidor
- utilitzar credencials predefinides per facilitar la connexió
- exposar el port **1433** per permetre connexions externes (per exemple, des de SSMS)

Un aspecte especialment rellevant és l'ús de volums:

- la carpeta **backup** del paquet es vincula al contenidor per facilitar l'accés als fitxers de còpia de seguretat
- un volum de Docker (sql_data) s'utilitza per emmagatzemar les dades de forma persistent

Aquesta configuració permet que la base de dades es mantinga entre diferents execucions del sistema.

Configuració del servei de l'API

El servei de l'API està definit per executar l'aplicació backend dins d'un contenidor Docker.

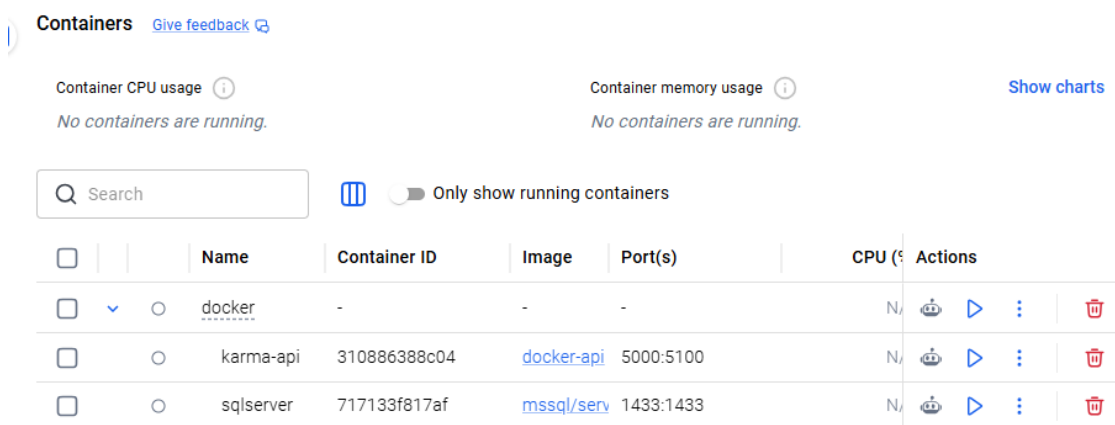
En aquest cas, no es compila el projecte dins del contenidor, sinó que es parteix d'una versió del backend ja compilada prèviament.

A través del Dockerfile, es configura un entorn mínim que:

- carrega els fitxers compilats de l'API
- executa l'aplicació mitjançant el runtime de .NET
- exposa un port per permetre l'accés des del navegador

A més, el fitxer docker-compose.yml estableix una dependència respecte a la base de dades, de manera que l'API s'inicia després del servei SQL Server.

Tot i això, és important destacar que aquesta dependència no garanteix que la base de dades estiga completament preparada, sinó només que el contenidor està en execució. Per aquest motiu, s'han implementat mecanismes de reintents en la connexió des de l'aplicació.



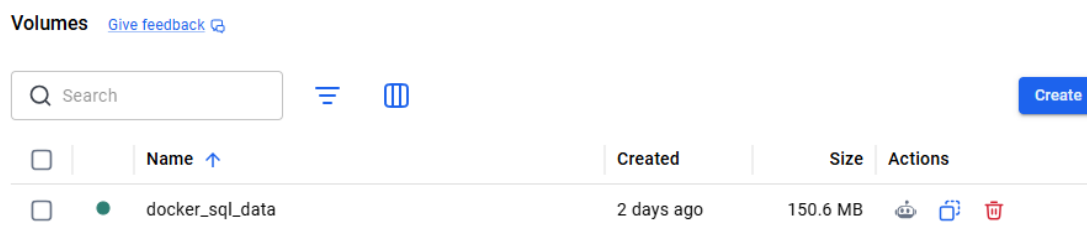
The screenshot shows the Docker Desktop interface for containers. At the top, there are sections for 'Container CPU usage' and 'Container memory usage', both indicating 'No containers are running.' with information icons. A 'Show charts' link is on the right. Below these is a search bar and a toggle for 'Only show running containers'. The main table lists three containers:

	Name	Container ID	Image	Port(s)	CPU (%)	Actions
☐	docker	-	-	-	N/A	
☐	karma-api	310886388c04	docker-api	5000:5100	N/A	
☐	sqlserver	717133f817af	mssql/serv	1433:1433	N/A	

Figura 2 Contenedors karma-api (api) i sqlserver (bd)

Volums i persistència de dades

El sistema utilitza un volum de Docker per gestionar la persistència de la base de dades.



The screenshot shows the Docker Desktop interface for volumes. At the top, there is a 'Volumes' header with a 'Give feedback' link. Below it is a search bar, a menu icon, and a 'Create' button. The main table lists one volume:

	Name	Created	Size	Actions
☐	docker_sql_data	2 days ago	150.6 MB	

Figura 3 Volum docker_sql_data

Aquest volum permet que les dades es conserven encara que els contenidors es paren o s'eliminen, evitant així la pèrdua d'informació entre sessions de treball.

Gràcies a aquest mecanisme, el funcionament és el següent:

- la primera vegada es crea la base de dades a partir del backup
- posteriorment, totes les dades es guarden de manera persistent
- en reiniciar el sistema, les dades continuen disponibles

En cas que es vulga reiniciar completament el sistema, és possible eliminar el volum, la qual cosa comporta la pèrdua de totes les dades i la necessitat de tornar a restaurar la base de dades inicial.

3.2. Dockerfile

El Dockerfile defineix el procés necessari per crear la imatge del contenidor de l'API.

En aquest projecte s'ha optat per una configuració simplificada en la qual:

- es parteix d'una imatge oficial de .NET (runtime)
- es copien directament els fitxers compilats de l'aplicació
- es defineix el punt d'entrada per executar la API

A diferència d'altres configuracions més complexes, no es realitza cap procés de compilació dins del contenidor.

```

1 # =====
2 # RUNTIME
3 # =====
4 FROM mcr.microsoft.com/dotnet/aspnet:8.0
5
6 WORKDIR /app
7
8 COPY api/ .
9
10 ENTRYPOINT ["dotnet", "KarmaWebAPI.dll"]

```

Figura 4 Contingut de Dockerfile

Execució de l'API a partir de binaris (.dll)

Una de les decisions més importants del disseny és la distribució del backend en forma de binaris compilats en lloc del codi font.

Aquest enfocament implica que:

- l'alumnat no té accés al codi intern del backend
- no és necessari disposar de l'entorn de desenvolupament .NET
- l'API es pot executar directament dins del contenidor

Aquest mecanisme contribueix a simplificar el sistema i a mantenir el focus en el desenvolupament del client.

Consideracions sobre el procés de build

Tot i que l'alumnat no realitza cap procés de compilació, aquest sí que és necessari per part del professorat **en el moment de preparar el paquet**.

Cada vegada que es vol distribuir un nou canvi en el codi del backend, és necessari generar una nova versió compilada de l'aplicació mitjançant l'eina de publicació de .NET.

Per a aquesta nova versió el docent ha de substituir el contingut de la carpeta **api**, que és la que s'utilitza posteriorment per executar el servei dins de Docker.

D'aquesta manera, Docker es limita a executar una versió ja preparada de l'aplicació, mentre que el procés de build es realitza externament en un entorn controlat.

3.3. Gestió de la base de dades

Ús del backup inicial (.bak)

El sistema utilitza una còpia de seguretat inicial de la base de dades, que el docent prepara i proporciona en format .bak dins de la carpeta **backup**.

Aquest fitxer representa un estat inicial del sistema, amb una estructura ja definida i, amb dades de prova que permeten començar a treballar a l'alumne immediatament.

Aquest enfocament evita que l'alumnat hagi de crear la base de dades manualment o executar scripts de creació complexos, simplificant així el procés inicial.

Procés de restauració inicial

En el moment en què es posa en marxa el sistema per primera vegada, la base de dades encara no existeix dins del contenidor de SQL Server.

Per aquest motiu, és necessari executar un procés de restauració que:

- crea la base de dades KarmaDB
- importa les dades des del fitxer .bak

Aquest procés es realitza mitjançant una comanda específica que executa l'script **init.sql** dins del contenidor (init.sql: es troba dins la carpeta backup de la distribució del paquet).

És important destacar que aquest pas:

- només s'ha de restaurar la base de dades la primera vegada
- no s'ha de repetir en execucions posteriors

Persistència de dades amb volums Docker

Una vegada creada, la base de dades es guarda dins d'un volum de Docker.

Aquest volum actua com un espai d'emmagatzematge persistent, independent del cicle de vida dels contenidors.

Gràcies a això:

- les dades es mantenen encara que el sistema es pare
- no es perden canvis entre sessions de treball
- no cal repetir la restauració inicial

Aquest mecanisme és essencial per proporcionar una experiència de treball continuada a l'alumnat.

Actualització del backup

El professorat pot actualitzar la base de dades inicial generant un nou fitxer .bak.

Aquest procediment permet:

- modificar l'estructura de la BD
- incorporar noves dades inicials
- adaptar el sistema a noves necessitats docents

Per aplicar aquesta actualització:

1. es substitueix el fitxer .bak en la carpeta backup
2. es reinicialitza el sistema eliminant el volum existent
3. es realitza novament el procés de restauració

D'aquesta manera, es pot distribuir una nova versió del backend amb una base de dades actualitzada.

4. Responsabilitats del professorat

4.1. Modificació del backend

El professorat és responsable de l'evolució del backend, ja que és qui disposa del codi font complet del projecte.

Quan es realitzen canvis en el backend, com per exemple:

- modificar la lògica dels serveis
- afegir nous endpoints
- corregir errors

és necessari generar una nova versió executable del projecte.

- **Procés de recompilació**

Per publicar una nova versió del backend, o bé es fa un Publish, o s'utilitza la comanda de .NET:

```
dotnet publish -c Release
```

El resultat és una carpeta “publish\” amb **tots els fitxers necessaris** per executar l'aplicació.

- **Actualització del paquet**

Una vegada generada la nova versió:

- es substitueix el contingut de la carpeta api
- es manté la mateixa estructura del projecte

Aquesta operació permet actualitzar el backend sense modificar la resta del sistema.

4.2. Actualització de la base de dades

Si el backend requereix canvis en la base de dades (per exemple, noves taules o camps), el professorat ha de generar un nou backup.

Aquest procés es realitza habitualment des de SQL Server:

- es creen els canvis necessaris
- es genera una còpia de seguretat actualitzada

Posteriorment, aquest fitxer substitueix el backup anterior del paquet.

- **Adaptació del sistema**

Una vegada actualitzat el backup:

- és necessari reinicialitzar la base de dades en el sistema Docker
- l'alumnat haurà de tornar a restaurar la BD per utilitzar la nova versió

Nota: Si el projecte està en fase de desenvolupament i l'alumnat té dades que no vol perdre, aquest haurà d'executar uns scripts d'actualització de la base de dades que hauran de ser proporcionats pel professorat.

4.3. Preparació i distribució del paquet

Una vegada actualitzats tant el backend com la base de dades, el professorat ha de preparar el paquet final per a l'alumnat.

Aquest procés inclou:

- verificar que la carpeta api conté la versió correcta

- assegurar que el backup de la base de dades és coherent amb el backend
- comprovar que el sistema funciona correctament en Docker
- **Validació en entorn net**

És especialment recomanable provar el sistema en una carpeta nova o amb els contenidors eliminats, simulant el comportament real d'un alumne.

Això permet detectar possibles errors abans de distribuir el paquet.

- **Distribució**

Finalment, el projecte s'ha de distribuir en github, com un paquet que incloga:

- els fitxers Docker
- el backend compilat
- el backup de la base de dades
- la documentació

Ruta:

<https://github.com/somarrod/Karma/tree/tfm/alumnat/recursos/docker>

5. Ús alternatiu amb Visual Studio

Encara que no és l'objectiu del mòdul DWEC, si algun alumne ho requereix, també li se pot proporcionar el codi del projecte KarmaWebApi, disponible en github en la ruta:

<https://github.com/somarrod/KarmaWebApi/tree/tfm>

Aquest projecte també inclou els fitxers necessaris per a fer la distribució en el contenidor de Docker.

5.1. Descripció del projecte amb codi font

A més del paquet simplificat basat en Docker, el projecte també està disponible en format complet amb el codi font del backend.

Aquest entorn inclou:

- el projecte ASP.NET complet
- la configuració de serveis, controladors i models
- la integració amb Entity Framework i SQL Server

Aquest format està orientat al desenvolupament i manteniment del backend, i no és necessari per a l'alumnat en el desenvolupament habitual.

5.2. Execució en entorn de desenvolupament

En aquest cas, el backend es pot executar directament des de Visual Studio o des de línia de comandes.

Per fer-ho, és necessari:

- disposar del SDK de .NET corresponent
- restaurar dependències del projecte
- executar l'aplicació

La connexió amb la base de dades es realitza mitjançant la cadena de connexió configurada en el projecte.

5.3. Connexió amb SQL Server

A diferència de l'entorn Docker, en aquest cas el desenvolupador pot:

- utilitzar un SQL Server instal·lat localment
- utilitzar el mateix contenidor Docker
- utilitzar altres entorns segons convinga

Això aporta flexibilitat, però al mateix temps implica una major responsabilitat en la configuració.

5.4. Diferències respecte a l'entorn Docker

Les diferències principals entre ambdós entorns són:

- En Docker, el backend es proporciona ja compilat
- En Visual Studio, el backend es desenvolupa i s'executa des del codi font
- Docker elimina problemes de configuració, mentre que Visual Studio permet un control més detallat

6. Possibles millores i ampliacions a futur

6.1. Automatització del procés d'inicialització

Actualment, la creació de la base de dades requereix una acció manual en la primera execució del sistema.

Una possible millora consistiria en automatitzar aquest procés, de manera que el sistema detecte si la base de dades existeix i, en cas contrari, execute automàticament la restauració.

Tot i que aquesta funcionalitat es pot implementar mitjançant scripts o mecanismes addicionals dins del contenidor, s'ha optat per una solució manual en aquest projecte per tal de garantir major control i evitar problemes derivats del moment d'arrancada del servidor SQL.

6.2. Scripts simplificats per a l'alumnat

Una altra millora interessant seria proporcionar scripts d'arrancada simplificats per a l'alumnat, amb l'objectiu de reduir encara més la complexitat d'ús.

Per exemple:

- un script per a arrancar el sistema
- un script per inicialitzar la base de dades
- un script per obrir automàticament Swagger

Aquests scripts podrien estar adaptats a diferents sistemes operatius (Windows i Linux/macOS), facilitant així la usabilitat, especialment en entorns educatius amb perfils no tècnics.

6.3. Empaquetat en màquina virtual

De cara a facilitar al màxim la posada en marxa del sistema, es podria proporcionar una màquina virtual preconfigurada amb:

- Docker instal·lat
- el projecte ja preparat
- la base de dades inicialitzada
- Microsoft SQL Server Management Studio ja instal·lat

Aquest enfocament eliminaria pràcticament qualsevol problema d'instal·lació o configuració en els equips de l'alumnat.

7. Annexos

7.1. Problemes habituals i solucions

L'API no s'inicia

Síntoma: no es pot accedir a Swagger.

Possibles causes:

- el contenidor de l'API no està en execució
- problema en l'arrancada inicial

Solució:

`docker-compose restart api`

o iniciar el contenidor manualment des de Docker Desktop.

Errors de connexió amb la base de dades

Síntoma: errors com "Login failed" o "Cannot open database".

Causa habitual:

- la base de dades no ha sigut restaurada o
- no troba la base de dades

Solució:

Executar el procés de restauració inicial.

Utilitzar SQL Management Studio per a validar l'accés a la base de dades

Problemes en la restauració inicial

Síntoma: el procés de restauració falla o no crea la BD.

Possibles causes:

- error en el fitxer .bak
- SQL Server encara no està preparat

Solució:

- esperar uns segons abans d'executar la restauració
- repetir el procés

Problemes de ports (1433)

Síntoma: no es pot connectar amb SQL Server des de fora.

Causa habitual:

- el port 1433 està ocupat

Solució:

- parar altres serveis SQL locals
- canviar el port en **docker-compose.yml**

Estat inconsistent del sistema

Síntoma: errors diversos després de múltiples execucions.

Solució recomanada:

```
docker-compose down -v
```

```
docker-compose up -d
```

reinicia completament el sistema.

7.2. Comandes útils de Docker

Arrancada i parada del sistema

```
docker-compose up -d
```

```
docker-compose down
```

Reinici de serveis

```
docker-compose restart
```

```
docker-compose restart api
```

Consulta de contenidors

Permet veure quins contenidors estan en execució.

```
docker ps
```

Consulta de logs

Permet consultar errors i problemes segons el contenidor.

```
docker logs karma-api
```

```
docker logs sqlserver
```

Execució de comandes dins del contenidor

```
docker exec -it sqlserver bash
```

o executar ordres específiques com restauració de BD.

Neteja de l'entorn

```
docker-compose down -v
```

```
docker system prune -f
```

Utilitzat per eliminar contenidors, volums i dades. Perdrà la BD i s'haurà de restaurar des de l'inici.